Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych
Inżynieria Oprogramowania

RoboFleet

Checkpoint 1: wymagania, funkcjonalności i struktura systemu

Historyjki użytkownika, diagramy przypadków użycia, opisy przypadków użycia i diagramy klas

Anna Granos
Mikołaj Zawrotniak
Patrick Fongang
Bartek Dźwigała

Październik 2026

Spis treści

1 Wprowadzenie i przegląd systemu

2 Historyjki użytkownika

3 Przypadki użycia

4 Kluczowe wymagania w szczegółach

5 Struktura systemu (diagramy klas)

6 Notacja i słownik pojęć

1 Wprowadzenie i przegląd systemu

1.1 Cel i zakres

RoboFleet pozwala **administratorowi** zaprojektować magazyn jako siatkę 2D (20 × 20 pól) z regałami, przeszkodami, stacjami odbioru i dostawy oraz pozycjami startowymi robotów. **Klient** tworzy zlecenia transportowe między stacjami i uruchamia symulację. Backend przydziela zlecenia robotom, planuje trasy bez kolizji dla 3–10 robotów, przesuwa je w dyskretnych krokach czasowych (*tickach*) i przesyła stan do przeglądarki, w której klient obserwuje flotę na żywo. Po zakończeniu przebiegu zapisywane są statystyki, takie jak czas wykonania, długości tras, rozwiązane konflikty i liczba wykonanych zleceń.

System jest wyłącznie symulacją: nie korzysta ze sprzętu, uczenia maszynowego ani płatnych usług zewnętrznych. Celem semestru jest kompletny, niezawodny system end-to-end, a nie globalnie optymalne planowanie tras.

1.2 Aktorzy

Aktor	Rodzaj	Opis
Gość	człowiek	Niezalogowany użytkownik. Może się zarejestrować i zalogować.
Zarejestrowany użytkownik	człowiek (abstrakcyjny)	Wspólny rodzic Klienta i Administratora. Przegląda magazyny, obserwuje przebiegi, przegląda wyniki, wylogowuje się.
Klient	człowiek	Tworzy przebiegi symulacji i zlecenia transportowe, uruchamia i zatrzymuje symulacje, analizuje i porównuje wyniki.
Administrator	człowiek	Zarządza użytkownikami, projektuje magazyny (siatka, stacje, roboty), ustawia parametry domyślne i scenariusze testowe.
Zegar symulacji	nie-ludzki (czas)	Timer w backendzie, który wyzwala każdy tick trwającej symulacji.

Sam backend **nie** jest aktorem, tylko projektowanym systemem. Jedynym aktorem nie-ludzkim jest czas (Zegar symulacji), który inicjuje przypadek użycia *Wykonaj krok symulacji*.

1.3 Moduły

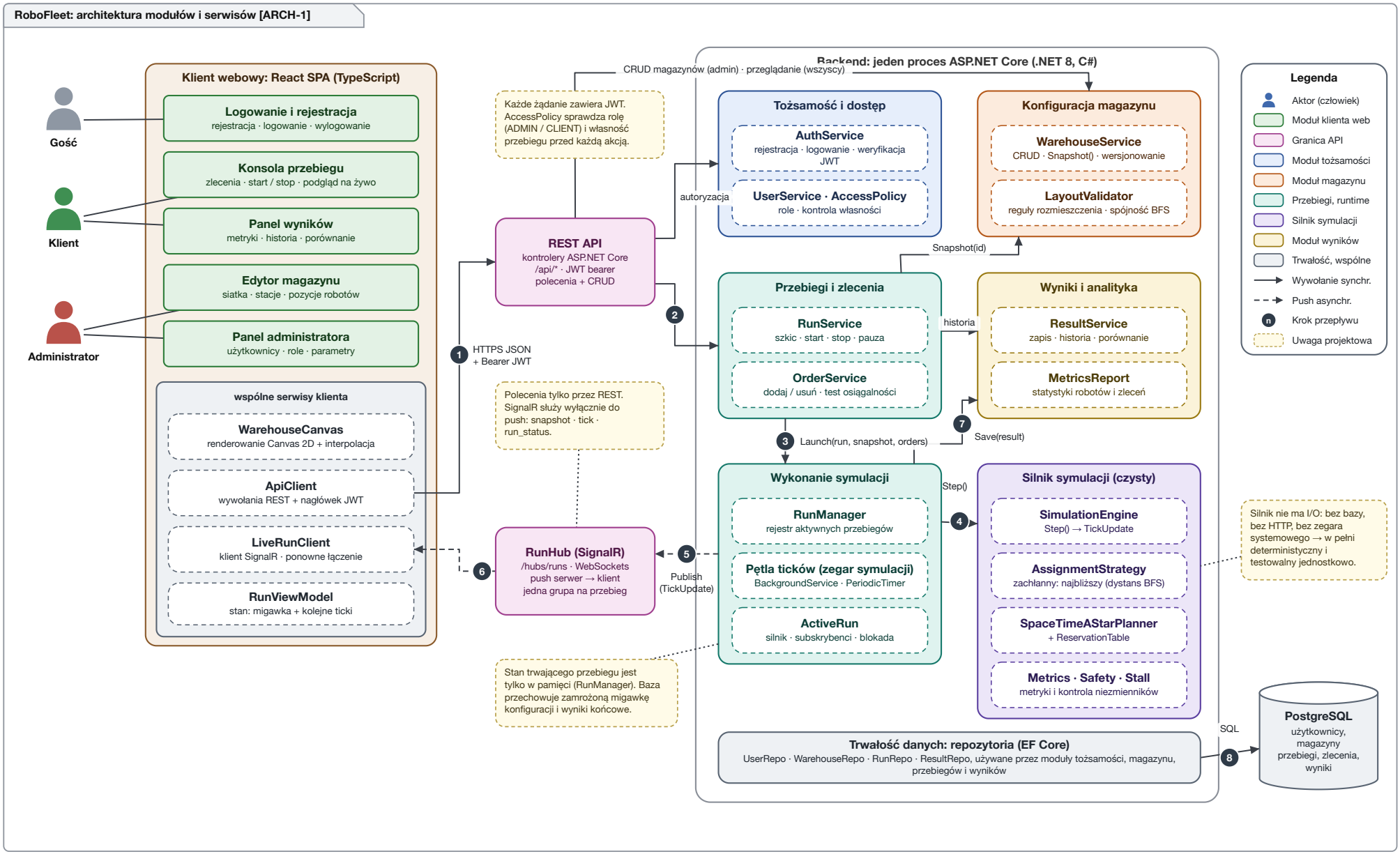
System jest podzielony na niezależne moduły. Moduły nie sięgają bezpośrednio do danych innych modułów; komunikują się wyłącznie przez interfejsy serwisów opisane w rozdziale 5.1. Rysunek ARCH-1 pokazuje moduły i główne przepływy.

Moduł	Odpowiedzialność	Główne serwisy
Klient webowy	Strony interfejsu, edytor siatki, podgląd 2D na żywo (Canvas), panel wyników.	ApiClient, LiveRunClient, WarehouseCanvas
Warstwa API	Kontrolery REST (polecenia, CRUD, zapytania) oraz hub SignalR (push z serwera).	ApiControllers, RunHub
Tożsamość i dostęp	Rejestracja, logowanie, JWT, role, kontrola własności, zarządzanie użytkownikami.	AuthService, UserService, AccessPolicy
Konfiguracja magazynu	CRUD magazynów, walidacja układu, migawki konfiguracji.	WarehouseService, LayoutValidator
Przebiegi i zlecenia	Szkice przebiegów, zlecenia transportowe, polecenia start / stop.	RunService

Moduł	Odpowiedzialność	Główne serwisy
Wykonanie symulacji	Aktywne przebiegi w pamięci, pętla ticków (Zegar symulacji), publikowanie na żywo.	RunManager, LiveUpdatePublisher
Silnik symulacji	Czysta, deterministyczna symulacja: przydział, planowanie tras, rezerwacje, metryki.	SimulationEngine
Wyniki i analityka	Zapis wyników, historia przebiegów, porównania.	ResultService
Trwałość danych	Repozytoria nad PostgreSQL (Entity Framework Core).	IRepository<T> i 4 implementacje

1.4 Założenia (obecny stan wiedzy)

- Zlecenia należą do **przebiegu symulacji**: klient tworzy szkic przebiegu, dodaje zlecenia i go uruchamia. Każde zlecenie jest wykonywane dokładnie raz.
- Przy starcie przebiegu system zapisuje **zamrożoną migawkę** magazynu i parametrów, więc późniejsze zmiany nie wpływają na trwające ani zakończone przebiegi.
- Czas jest dyskretny: w jednym ticku każdy robot przesuwa się o jedno pole (sąsiedztwo 4-kierunkowe) albo czeka. Załadunek i rozładunek trwają konfigurowalną liczbę ticków.
- Przydział (MVP): oczekujące zlecenia w kolejności FIFO trafiają do najbliższego wolnego robota; odległość to rzeczywista długość ścieżki (BFS), a nie linia prosta.
- Koordynacja (MVP): planowanie priorytetowe z algorytmem space-time A* i tablicą rezerwacji. Z konstrukcji zapobiega konfliktom wierzchołkowym i krawędziowym (zamianie miejsc). **Nie** gwarantuje optymalnych tras ani braku zakleszczeń; zakleszczenia są wykrywane (brak postępu przez K ticków) i raportowane.
- Po dostarczeniu robot wraca do swojej pozycji startowej. Bezczynne roboty nadal zajmują swoje pole.
- Samodzielna rejestracja tworzy konto Klienta; pierwszy Administrator jest tworzony przy wdrożeniu.
- Jeden proces backendu (ASP.NET Core). Pętla ticków działa jako hostowany `BackgroundService`; stan na żywo jest w pamięci; PostgreSQL przechowuje konfiguracje, przebiegi, zlecenia i wyniki.
- Technologie: C# / .NET 8, ASP.NET Core, SignalR, Entity Framework Core, PostgreSQL, xUnit; po stronie klienta React + TypeScript z Canvas 2D; Docker Compose.



Rysunek ARCH-1. Moduły i serwisy. Numery pokazują główny przepływ przebiegu symulacji: (1) polecenie REST, (2) serwis przebiegów, (3) uruchomienie, (4) krok silnika w każdym ticku, (5) publikacja, (6) push SignalR do przeglądarki, (7) zapis wyniku, (8) SQL.

2 Historyjki użytkownika

2.1 Priorytetyzacja

Poniższe 29 historyjek pogrupowano według modułów i nadano im priorytety metodą MoSCoW: **Must** bez tego system nie ma sensu lub jest niebezpieczny · **Should** bardzo uciążliwy brak, ale system jest używalny · Could warto mieć · Won't poza zakresem w tym semestrze. Identyfikatory mają postać `moduł.numer`; te same identyfikatory występują w tabeli śledzenia (rozdział 3.6) i na kartach historyjek (rozdział 4.1).

2.2 Lista historyjek

01 · Tożsamość i dostęp

ID	Jako…	chcę…	aby…	Priorytet
01.01	Gość	zarejestrować konto przy użyciu e-maila i hasła	móc tworzyć zlecenia i uruchamiać symulacje	**Must**
01.02	Gość	zalogować się	mieć bezpieczny dostęp do swoich magazynów i przebiegów	**Must**
01.03	Zarejestrowany użytkownik	wylogować się	nikt inny nie mógł użyć mojej sesji na wspólnym komputerze	**Must**
01.04	Administrator	zobaczyć listę użytkowników i zmienić ich rolę	tylko zaufane osoby mogły edytować konfigurację magazynów	**Should**
01.05	Administrator	dezaktywować konto użytkownika	osoby, które odeszły z zespołu, straciły dostęp	**Should**

02 · Konfiguracja magazynu

ID	Jako…	chcę…	aby…	Priorytet
02.01	Administrator	utworzyć nowy magazyn jako siatkę 20 × 20	móc odwzorować nowy układ magazynu	**Must**
02.02	Administrator	rysować regały i przeszkody na siatce	układ odpowiadał rzeczywistemu magazynowi	**Must**
02.03	Administrator	umieszczać stacje odbioru i dostawy	klienci mogli definiować zlecenia między nimi	**Must**
02.04	Administrator	umieścić 1–10 robotów w ich pozycjach startowych	flota miała jasno określone punkty startu	**Must**
02.05	Administrator	otrzymać wynik walidacji przed zapisem układu	błędne układy nie trafiały do klientów	**Must**
02.06	Administrator	edytować i archiwizować zapisane magazyny	konfiguracje dało się ponownie wykorzystać, a lista była uporządkowana	**Must**
02.07	Administrator	ustawić domyślne parametry symulacji (czas obsługi, limit ticków, prędkość, strategia)	przebiegi w danym magazynie działały spójnie	**Should**

ID	Jako…	chcę…	aby…	Priorytet
02.08	Administrator	definiować scenariusze testowe (stały zestaw zleceń i parametrów)	eksperymenty dało się dokładnie powtórzyć	Could
02.09	Zarejestrowany użytkownik	przeglądać magazyny i podglądać ich układ	wybrać magazyn do symulacji	**Must**

03 · Przebiegi i symulacja

ID	Jako…	chcę…	aby…	Priorytet
03.01	Klient	utworzyć przebieg symulacji dla wybranego magazynu	przygotować eksperyment	**Must**
03.02	Klient	dodawać zlecenia transportowe (odbiór → dostawa, opcjonalny tick zwolnienia)	flota miała pracę do wykonania	**Must**
03.03	Klient	usuwać zlecenia przed startem przebiegu	poprawić błędy w konfiguracji	**Must**
03.04	Klient	uruchomić symulację z automatycznym przydziałem zleceń i planowaniem tras	nie planować pracy robotów ręcznie	**Must**
03.05	Zarejestrowany użytkownik	obserwować ruch robotów na żywo na mapie 2D wraz ze statusami zleceń	rozumieć zachowanie floty	**Must**
03.06	Klient	zatrzymać trwającą symulację	przerwać eksperyment, który przestał być przydatny	**Must**
03.07	Klient	zobaczyć, dlaczego zlecenie się nie powiodło (nieosiągalne, limit czasu, zakleszczenie)	poprawić układ magazynu lub zlecenia	**Must**
03.08	Klient	wstrzymywać i wznawiać symulację oraz zmieniać jej prędkość	dokładnie przeanalizować ciekawe momenty	**Should**
03.09	Zarejestrowany użytkownik	wrócić do trwającej symulacji po odświeżeniu strony	problem z siecią nie psuł prezentacji	**Should**
03.10	Klient	dodawać zlecenia w trakcie trwania symulacji	testować dynamiczne obciążenie	Could

04 · Wyniki i analityka

ID	Jako…	chcę…	aby…	Priorytet
04.01	Klient	zobaczyć statystyki po zakończeniu przebiegu	ocenić efektywność pracy floty	**Must**
04.02	Zarejestrowany użytkownik	przeglądać historię przebiegów i otwierać zapisane wyniki	wracać do wcześniejszych eksperymentów	**Must**
04.03	Klient	sklonować przebieg z innymi parametrami (np. strategią przydziału)	testować warianty na identycznych zleceniach	Could
04.04	Klient	porównać dwa przebiegi obok siebie	zobaczyć, która strategia działa lepiej	Could
04.05	Administrator	widzieć przebiegi wszystkich użytkowników	wspierać klientów i diagnozować problemy	Could

2.3 Poza zakresem (w tym semestrze)

- Integracja z fizycznymi robotami lub sprzętem; optymalizacja z użyciem AI / uczenia maszynowego; płatne zewnętrzne API.
- Baterie i ładowanie, roboty o różnych prędkościach lub ładowności, ruch po przekątnej.
- Optymalne planowanie wieloagentowe (np. CBS) i gwarantowany brak zakleszczeń.
- Wizualizacja 3D, aplikacja mobilna, magazyny większe niż ok. 30 × 30 pól, skalowanie horyzontalne.

2.4 Wymagania niefunkcjonalne

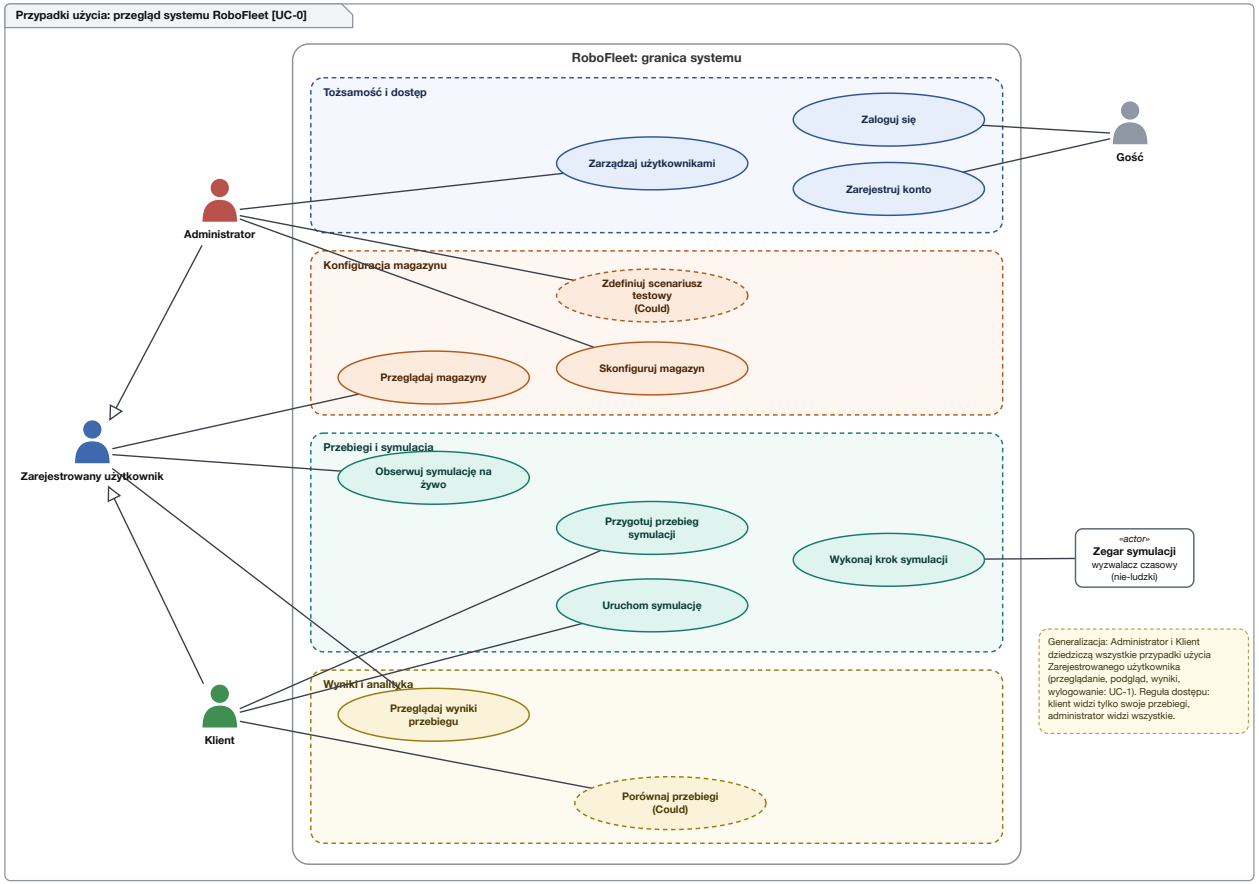
ID	Kategoria	Wymaganie	Weryfikacja
NFR-01	Niezawodność	Dwa roboty nigdy nie zajmują tego samego pola ani nie zamieniają się polami w tym samym ticku. Monitor bezpieczeństwa sprawdza to w każdym ticku; raportowana liczba kolizji musi wynosić 0.	Sprawdzenie niezmienników w każdym teście i przebiegu
NFR-02	Niezawodność	Każdy przebieg się kończy: wszystkie zlecenia zakończone, osiągnięty limit ticków albo brak postępu przez K ticków (zastój).	Testy losowe silnika
NFR-03	Niezawodność	Symulacja jest deterministyczna: ta sama migawka, zlecenia i parametry dają identyczne wyniki.	Dwukrotne uruchomienie, porównanie wyniku
NFR-04	Wydajność	Jeden tick dla 10 robotów na siatce 20 × 20 jest liczony w mniej niż 100 ms (cel, mierzony).	Pomiar w trybie bez interfejsu
NFR-05	Wydajność / użyteczność	Podgląd na żywo odświeża się co najmniej dwa razy na sekundę, z interpolacją ruchu robotów.	Kontrola podczas demo
NFR-06	Bezpieczeństwo	Hasła są przechowywane jako skróty (hasher ASP.NET Core Identity). Każde wywołanie REST i każde połączenie SignalR jest uwierzytelniane tokenem JWT; rola i własność są sprawdzane po stronie serwera.	Testy API dla 401 / 403
NFR-07	Użyteczność	Błędy walidacji układu są podświetlane na siatce; każda metryka ma podpowiedź z definicją.	Przegląd interfejsu
NFR-08	Utrzymywalność	Cały system uruchamia się poleceniem `docker compose up`, korzysta tylko z darmowych narzędzi open source, a CI uruchamia testy.	Pipeline CI
NFR-09	Testowalność	Silnik symulacji nie wykonuje operacji wejścia/wyjścia (baza, HTTP, zegar systemowy) i może być testowany w izolacji.	Testy jednostkowe silnika

3 Przypadki użycia

Diagramy przypadków użycia grupują funkcjonalność według modułów; każdy podrozdział zaczyna się krótkim wstępem i diagramem. Elipsy mają kolor modułu, elipsy przerywane to przypadki o priorytecie *Could*. «include» oznacza obowiązkową część innego przypadku użycia, «extend» opcjonalną (jej warunek jest opisany jako punkt rozszerzenia w notatce).

3.1 Przegląd

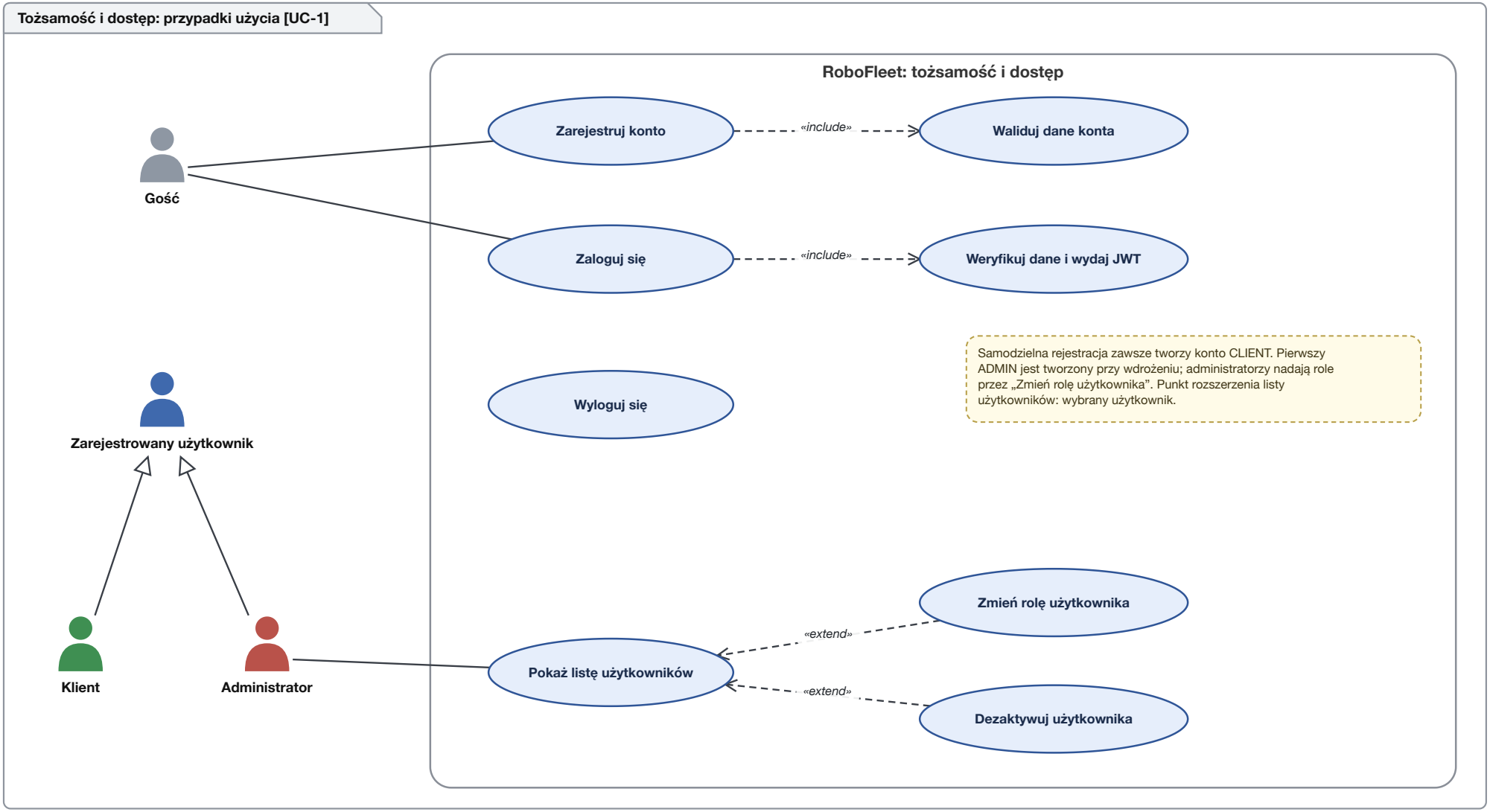
Przegląd pokazuje granicę systemu, cztery moduły funkcjonalne jako pakiety oraz wszystkich aktorów. Administrator i Klient dziedziczą po Zarejestrowanym użytkowniku, co pokazuje ich wspólne funkcje: przeglądanie magazynów, obserwację symulacji, przeglądanie wyników i wylogowanie. Aktor nie-ludzki Zegar symulacji wyzwala każdy tick. Diagram nie modeluje kolejności przypadków użycia; przebieg symulacji opisano w rozdziale 4.2.



Rysunek UC-0. Przegląd przypadków użycia systemu RoboFleet.

3.2 Tożsamość i dostęp

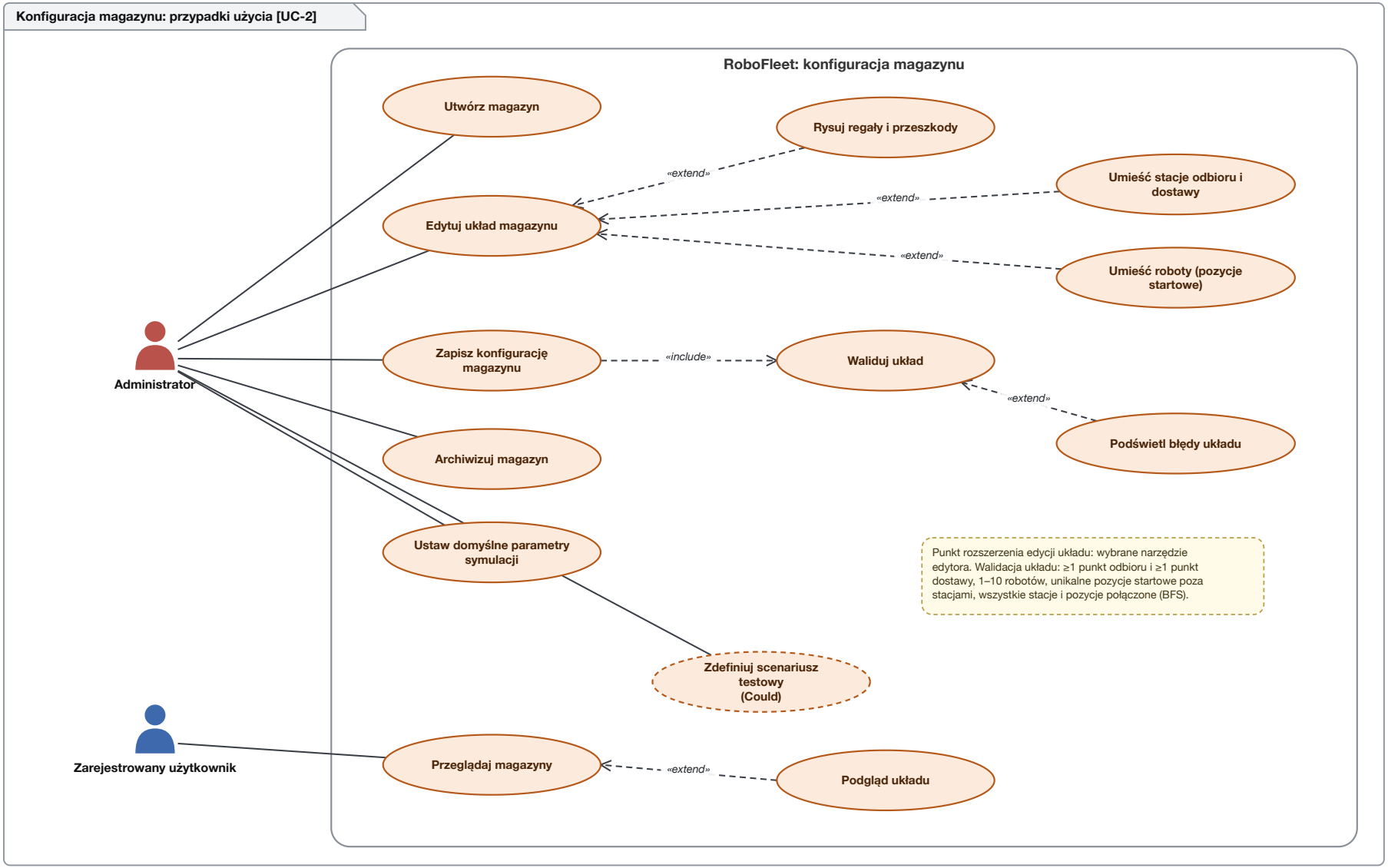
Gość rejestruje się i loguje; każdy zarejestrowany użytkownik może się wylogować. Logowanie zawsze obejmuje weryfikację danych i wydanie tokenu JWT; rejestracja zawsze obejmuje walidację danych konta. Administrator przegląda listę użytkowników i opcjonalnie zmienia rolę lub dezaktywuje użytkownika.



Rysunek UC-1. Przypadki użycia: tożsamość i dostęp.

3.3 Konfiguracja magazynu

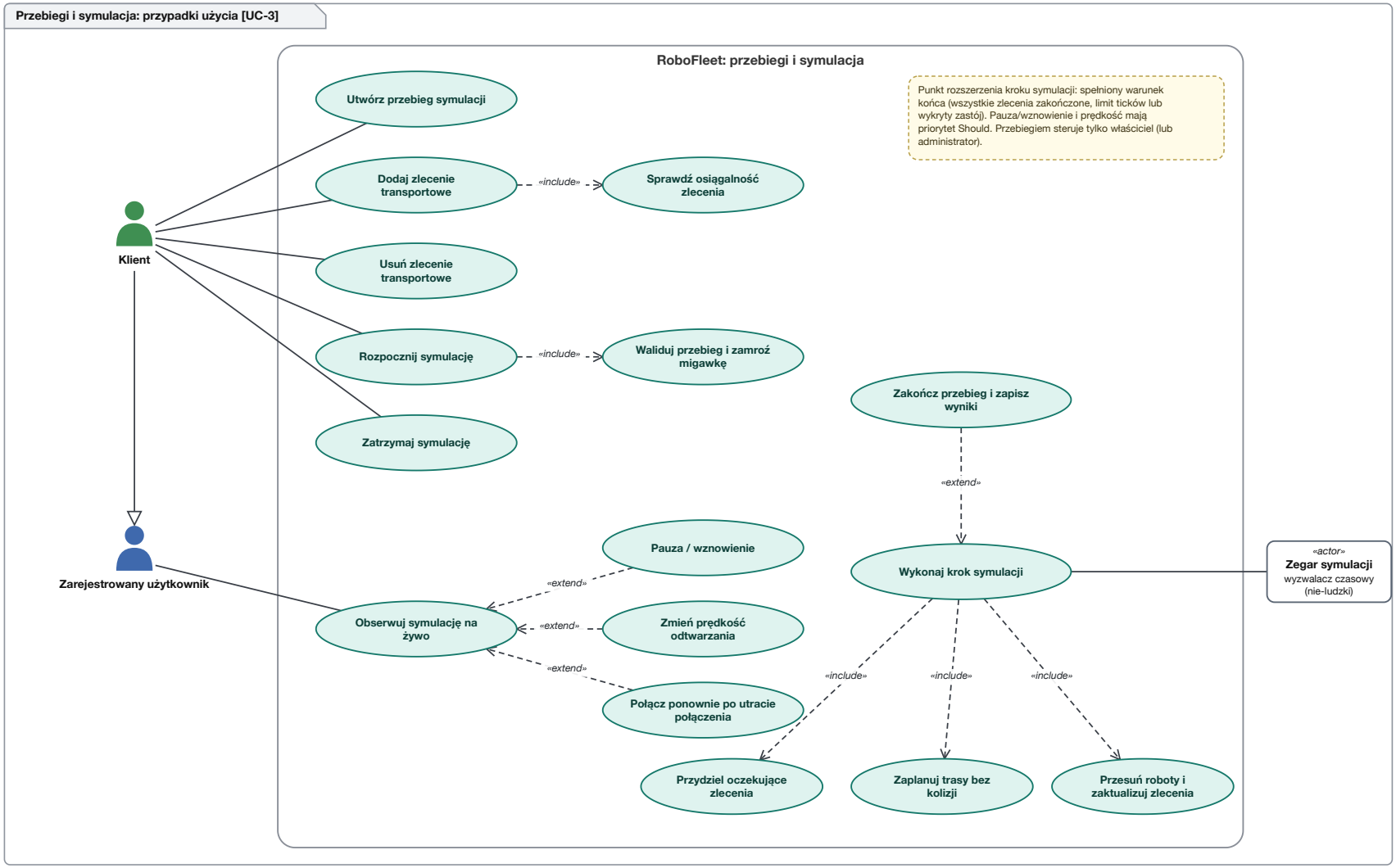
Administrator tworzy, edytuje, zapisuje i archiwizuje magazyny. Edycja jest rozszerzana przez trzy narzędzia edytora (regały i przeszkody, stacje, roboty). Zapis zawsze obejmuje walidację; dla niepoprawnych układów walidacja jest rozszerzana o podświetlenie błędów. Każdy zarejestrowany użytkownik może przeglądać magazyny i podglądać ich układ.



Rysunek UC-2. Przypadki użycia: konfiguracja magazynu.

3.4 Przebiegi i symulacja

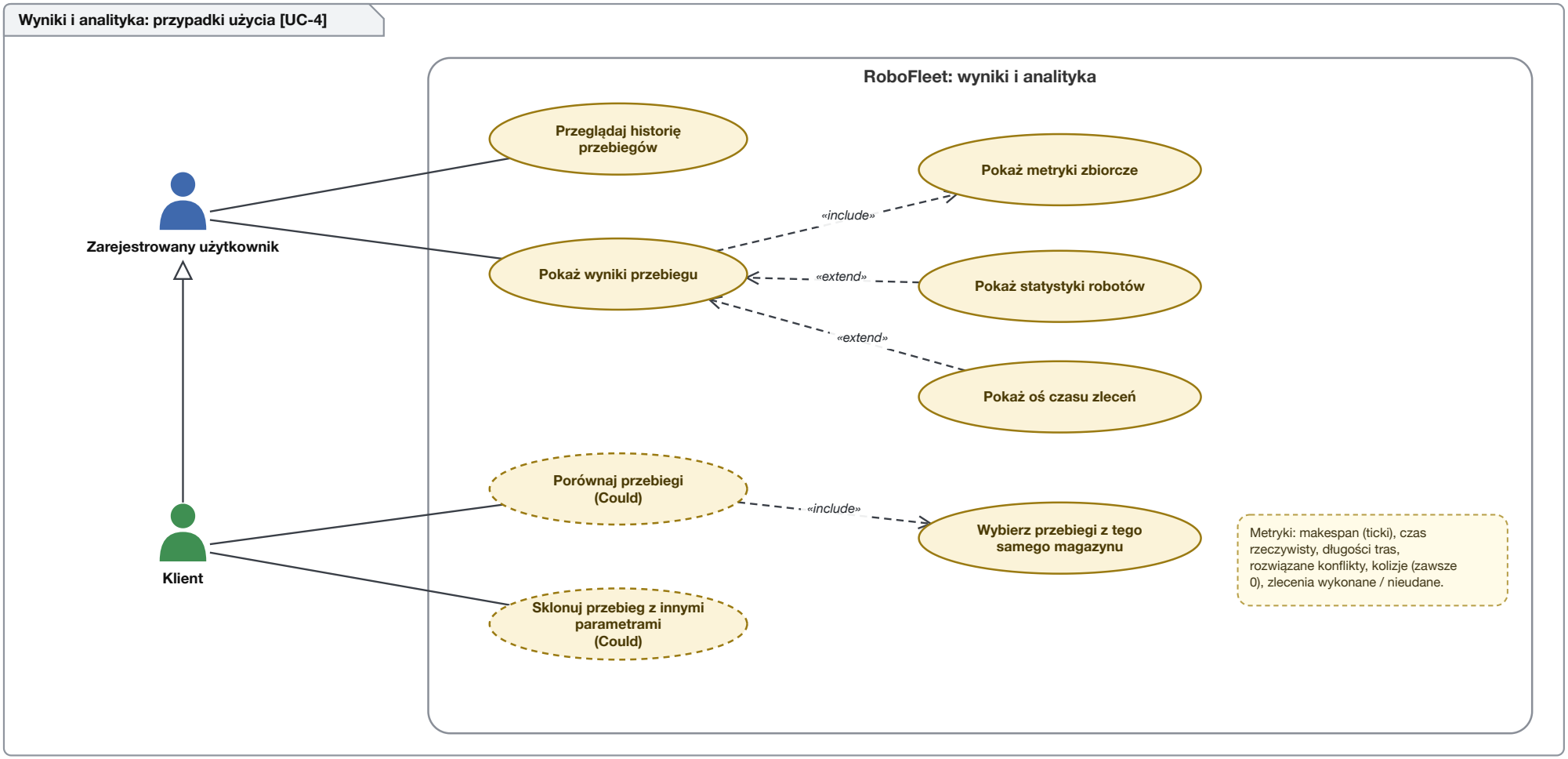
Klient przygotowuje przebiegi (tworzy, dodaje i usuwa zlecenia) oraz je uruchamia i zatrzymuje. Dodanie zlecenia zawsze obejmuje sprawdzenie osiągalności; start zawsze obejmuje walidację przebiegu i zamrożenie migawki konfiguracji. Obserwacja przebiegu może być rozszerzona o pauzę i wznowienie, zmianę prędkości oraz ponowne połączenie. Zegar symulacji wykonuje ticki; każdy tick obejmuje przydział zleceń, planowanie tras i ruch robotów, a zakończenie przebiegu jest rozszerzeniem uruchamianym po spełnieniu warunku końca.



Rysunek UC-3. Przypadki użycia: przebiegi i symulacja.

3.5 Wyniki i analityka

Każdy zarejestrowany użytkownik przegląda historię swoich przebiegów i otwiera wyniki (administrator widzi wszystkie przebiegi). Pokazanie wyników zawsze obejmuje metryki zbiorcze i może być rozszerzone o statystyki robotów oraz oś czasu zleceń. Klonowanie i porównywanie przebiegów to funkcje o priorytecie Could, służące do porównywania strategii na identycznych zleceniach.



Rysunek UC-4. Przypadki użycia: wyniki i analityka.

3.6 Śledzenie: przypadki użycia ↔ historyjki użytkownika

Przypadek(i) użycia	**Historyjki**
UC-1 Zarejestruj konto · Zaloguj się · Wyloguj się	01.01, 01.02, 01.03
UC-1 Pokaż listę użytkowników «extend» Zmień rolę / Dezaktywuj	01.04, 01.05
UC-2 Utwórz magazyn · Edytuj układ (+ regały / stacje / roboty)	02.01–02.04
UC-2 Zapisz konfigurację «include» Waliduj układ	02.05, 02.06
UC-2 Ustaw parametry domyślne · Zdefiniuj scenariusz testowy	02.07, 02.08
UC-2 Przeglądaj magazyny «extend» Podgląd układu	02.09
UC-3 Utwórz przebieg · Dodaj / usuń zlecenie	03.01–03.03
UC-3 Rozpocznij symulację · Wykonaj krok symulacji	03.04, 03.07
UC-3 Obserwuj symulację na żywo (+ pauza, prędkość, ponowne połączenie)	03.05, 03.08, 03.09
UC-3 Zatrzymaj symulację	03.06
UC-4 Pokaż wyniki przebiegu (+ roboty, oś czasu zleceń)	04.01
UC-4 Przeglądaj historię przebiegów	04.02, 04.05
UC-4 Sklonuj przebieg · Porównaj przebiegi	04.03, 04.04

4 Kluczowe wymagania w szczegółach

4.1 Karty historyjek (rewers)

Dla czterech najważniejszych historyjek o priorytecie Must przedstawiono awers karty (rola, funkcja, uzasadnienie) oraz rewers z listą kryteriów akceptacji typu tak / nie i wymaganiami niefunkcjonalnymi.

ID: 02.05		**TYTUŁ: Walidacja układu magazynu**
Jako…	**chcę…**	**aby…**
Administrator	otrzymać wynik walidacji przed zapisem układu	błędne układy nie trafiały do klientów

Lista kryteriów akceptacji *(funkcjonalność)*

- Czy zapis jest blokowany, gdy układ nie ma stacji odbioru lub dostawy?
- Czy stacje i pozycje robotów, które nie są ze sobą połączone, są podświetlane?
- Czy pozycja robota na regale, na stacji lub współdzielona z innym robotem jest odrzucana?
- Czy liczba robotów jest ograniczona do 1–10?
- Czy ostrzeżenia (np. pozycja robota w korytarzu o szerokości 1 pola) są pokazywane, ale zapis jest możliwy?

Wymagania niefunkcjonalne *(Użyteczność · Niezawodność · Wydajność · Bezpieczeństwo)*

- Czy błędne pola są podświetlane bezpośrednio na siatce z czytelnym komunikatem? (U)
- Czy późniejszy zapis innego administratora nie nadpisuje po cichu mojego (kontrola wersji)? (N)
- Czy walidacja układu 20×20 trwa krócej niż 200 ms? (W)
- Czy tylko administratorzy mogą zapisywać układy magazynów? (B)

ID: 03.04 **TYTUŁ: Automatyczny przydział i planowanie tras**

Jako…	chcę…	aby…
Klient	uruchomić symulację z automatycznym przydziałem zleceń i planowaniem tras	nie planować pracy robotów ręcznie

Lista kryteriów akceptacji *(funkcjonalność)*

- Czy można uruchomić tylko przebieg z co najmniej jednym zleceniem?
- Czy każde zlecenie trafia do najbliższego wolnego robota (rzeczywista długość ścieżki, nie linia prosta)?
- Czy zlecenia czekają w kolejności FIFO, gdy wszystkie roboty są zajęte?
- Czy robot czeka lub planuje ponownie zamiast wjechać na zajęte pole?
- Czy przebieg kończy się automatycznie, gdy każde zlecenie jest wykonane lub nieudane?

Wymagania niefunkcjonalne *(Użyteczność · Niezawodność · Wydajność · Bezpieczeństwo)*

- Czy liczba kolizji (wspólne pole lub zamiana miejsc) zawsze wynosi 0? (N)
- Czy dwa przebiegi z identycznymi danymi dają identyczne wyniki? (N)
- Czy jeden tick dla 10 robotów trwa krócej niż 100 ms? (W)
- Czy przebieg może uruchomić lub zatrzymać tylko jego właściciel (lub administrator)? (B)

ID: 03.05 **TYTUŁ: Obserwacja symulacji na żywo**

Jako…	chcę…	aby…
Zarejestrowany użytkownik	obserwować ruch robotów na żywo na mapie 2D wraz ze statusami zleceń	rozumieć zachowanie floty

Lista kryteriów akceptacji *(funkcjonalność)*

- Czy widzę pozycję każdego robota, jego stan i to, czy wiezie ładunek?
- Czy widzę status każdego zlecenia i przypisanego do niego robota?
- Czy liczniki na żywo (tick, zlecenia wykonane / nieudane, rozwiązane konflikty) się aktualizują?
- Czy po odświeżeniu strony podgląd kontynuuje od bieżącego ticku?

Wymagania niefunkcjonalne *(Użyteczność · Niezawodność · Wydajność · Bezpieczeństwo)*

- Czy mapa odświeża się co najmniej dwa razy na sekundę z płynnym ruchem? (W/U)
- Czy roboty da się odróżnić kolorem i etykietą, z legendą? (U)
- Czy zamknięcie przeglądarki nie zatrzymuje symulacji na serwerze? (N)
- Czy inny klient nie może otworzyć mojego przebiegu? (B)

ID: 04.01		**TYTUŁ: Statystyki przebiegu**
Jako…	**chcę…**	**aby…**
Klient	zobaczyć statystyki po zakończeniu przebiegu	ocenić efektywność pracy floty

Lista kryteriów akceptacji *(funkcjonalność)*

- Czy widzę czas symulacji (ticki) oddzielnie od czasu rzeczywistego?
- Czy widzę zlecenia wykonane i nieudane, z przyczyną każdego niepowodzenia?
- Czy widzę planowaną i faktycznie przejechaną długość trasy dla każdego robota?
- Czy widzę rozwiązane konflikty i ticki opóźnienia, oddzielnie od kolizji (zawsze 0)?
- Czy widzę wykorzystanie każdego robota?

Wymagania niefunkcjonalne *(Użyteczność · Niezawodność · Wydajność · Bezpieczeństwo)*

- Czy wyniki są dostępne po wylogowaniu i restarcie serwera? (N)
- Czy późniejsza edycja magazynu nie zmienia starych wyników (migawka)? (N)
- Czy strona wyników ładuje się w mniej niż 1 sekundę? (W)
- Czy każda metryka pokazuje swoją definicję w podpowiedzi? (U)

4.2 Opisy przypadków użycia (szablon Cockburna)

Trzy kluczowe przypadki użycia opisano zgodnie ze strukturą zaproponowaną przez Alistaira Cockburna (UML @ Classroom, tabela 3.1).

UC-3.4 Uruchomienie i przebieg symulacji

Nazwa	Uruchomienie i przebieg symulacji
Krótki opis	Klient uruchamia przygotowany przebieg. System zamraża konfigurację, wykonuje kolejne ticki, przesyła stan do przeglądarki i zapisuje wyniki po zakończeniu przebiegu.
Warunek wstępny	Klient jest zalogowany i jest właścicielem przebiegu. Przebieg ma status DRAFT i co najmniej jedno zlecenie. Magazyn jest poprawny. Limit aktywnych przebiegów nie jest przekroczony.
Warunek końcowy	Przebieg ma status końcowy (COMPLETED, FAILED lub STOPPED). Każde zlecenie ma status końcowy. Wyniki są zapisane i widoczne w historii przebiegów.
Sytuacje błędne	E1 przebieg już uruchomiony (nie DRAFT) · E2 przebieg bez zleceń · E3 osiągnięty limit aktywnych przebiegów · E4 brak postępu przez K ticków (zakleszczenie / zastój) · E5 osiągnięty maksymalny limit ticków · E6 naruszony niezmiennik bezpieczeństwa (błąd silnika) · E7 restart serwera w trakcie przebiegu.
Stan systemu w razie błędu	E1–E3: nic się nie zmienia, przebieg pozostaje w stanie DRAFT. E4–E5: przebieg przechodzi w stan FAILED z przyczyną, niezakończone zlecenia otrzymują status FAILED, częściowe wyniki są zapisywane. E6: przebieg FAILED (ENGINE_ERROR), wyniki są zachowane do diagnozy. E7: przebieg przechodzi w stan INTERRUPTED.
Aktorzy	Klient (główny), Zegar symulacji (pomocniczy)
Wyzwalacz	Klient naciska **Start** w konsoli przebiegu.
Scenariusz główny	1. Klient naciska Start. 2. System sprawdza własność, status przebiegu i zlecenia. 3. System zamraża migawkę magazynu i parametrów oraz ustawia status RUNNING. 4. Przeglądarka otwiera kanał na żywo; system wysyła początkową migawkę. 5. Przy każdym ticku zegara system wykonuje jeden krok symulacji (UC-3.T) i wysyła aktualizację. 6. Krok 5 powtarza się, aż każde zlecenie zostanie wykonane lub zakończy się niepowodzeniem. 7. System ustawia status COMPLETED, buduje i zapisuje wyniki. 8. System pokazuje klientowi podsumowanie wyników.
Scenariusze alternatywne	2′ Kontrola nie przechodzi → komunikat o błędzie, przebieg pozostaje w stanie DRAFT. 5′ Klient naciska Stop → otwarte zlecenia otrzymują status CANCELLED, przebieg STOPPED, częściowe wyniki są zapisywane. 5″ Przeglądarka traci połączenie → symulacja trwa dalej; po ponownym połączeniu system wysyła bieżącą migawkę. 6′ Wykryty zastój lub limit ticków → przebieg FAILED z przyczyną.

UC-3.T Wykonanie kroku symulacji

Nazwa	Wykonanie kroku symulacji
Krótki opis	Symulacja przesuwa się o jeden dyskretny krok: zwolnienie zleceń, przydział, planowanie tras bez kolizji, ruch robotów i kontrola bezpieczeństwa.
Warunek wstępny	Przebieg ma status RUNNING (nie jest wstrzymany), a jego silnik jest zainicjalizowany.
Warunek końcowy	Licznik ticków zwiększony o 1. Żadne dwa roboty nie zajmują tego samego pola i nie zamieniły się miejscami. Stany robotów i zleceń są zaktualizowane, aktualizacja została wysłana.
Sytuacje błędne	E1 brak trasy dla robota · E2 stacja docelowa zajęta · E3 naruszony niezmiennik bezpieczeństwa · E4 brak postępu przez K kolejnych ticków.
Stan systemu w razie błędu	E1/E2: robot pozostaje na swoim (zarezerwowanym) polu i ponawia próbę w następnym ticku; po Y nieudanych próbach na stacji najpierw wraca do pozycji startowej. E3: przebieg FAILED (ENGINE_ERROR). E4: przebieg FAILED (STALLED).
Aktorzy	Zegar symulacji (nie-ludzki, główny)
Wyzwalacz	Upływa interwał timera (interwał ticku ÷ prędkość).
Scenariusz główny	1. Zwolnienie zleceń, których tick zwolnienia został osiągnięty. 2. Przydział oczekujących zleceń w kolejności FIFO do najbliższego wolnego robota (odległość ścieżki wg BFS). 3. Dla każdego robota bez trasy: planowanie trasy algorytmem space-time A* względem tablicy rezerwacji i jej zarezerwowanie. 4. Każdy robot wykonuje kolejny krok (ruch lub oczekiwanie). 5. Roboty na stacjach ładują / rozładowują; statusy zleceń są aktualizowane; roboty bez pracy wracają do pozycji startowych. 6. Kontrola niezmienników bezpieczeństwa i aktualizacja metryk. 7. Sprawdzenie warunków zakończenia. 8. Wysłanie aktualizacji ticku.
Scenariusze alternatywne	2′ Brak wolnego robota → zlecenie pozostaje PENDING. 3′ Brak trasy → robot czeka i ponawia próbę (E1). 7′ Wszystkie zlecenia zakończone → koniec przebiegu (COMPLETED). 7″ Limit ticków → FAILED (TIMEOUT); brak postępu przez K ticków → FAILED (STALLED).

UC-2.3 Zapis konfiguracji magazynu

Nazwa	Zapis konfiguracji magazynu
Krótki opis	Administrator zapisuje edytowany układ magazynu. System go waliduje i zapisuje nową wersję.
Warunek wstępny	Administrator jest zalogowany i ma otwarty w edytorze szkic układu.
Warunek końcowy	Magazyn jest zapisany z wersją zwiększoną o 1 i jest widoczny dla klientów. Istniejące przebiegi nie są zmieniane (używają migawek).
Sytuacje błędne	E1 niepoprawny układ (brak stacji odbioru / dostawy, nieosiągalna stacja, pozycja robota na regale lub stacji, zduplikowane pozycje, liczba robotów poza zakresem 1–10) · E2 inny administrator zapisał nowszą wersję · E3 wygasła sesja.
Stan systemu w razie błędu	Zapisany magazyn pozostaje bez zmian. Szkic pozostaje w edytorze z podświetlonymi błędami.
Aktorzy	Administrator
Wyzwalacz	Administrator naciska **Zapisz** w edytorze magazynu.
Scenariusz główny	1. Administrator naciska Zapisz. 2. System waliduje pola, stacje, pozycje robotów i spójność (BFS). 3. System nie zgłasza błędów. 4. System sprawdza, czy zapisana wersja jest równa wersji szkicu. 5. System zapisuje układ, stacje i roboty oraz zwiększa wersję. 6. System potwierdza zapis.
Scenariusze alternatywne	3′ Wykryto błędy → są podświetlane na siatce; administrator je poprawia i zapisuje ponownie. 3″ Tylko ostrzeżenia → administrator potwierdza „zapisz mimo to”. 4′ Konflikt wersji → system proponuje ponowne wczytanie; szkic zostaje zachowany do porównania.

5 Struktura systemu (diagramy klas)

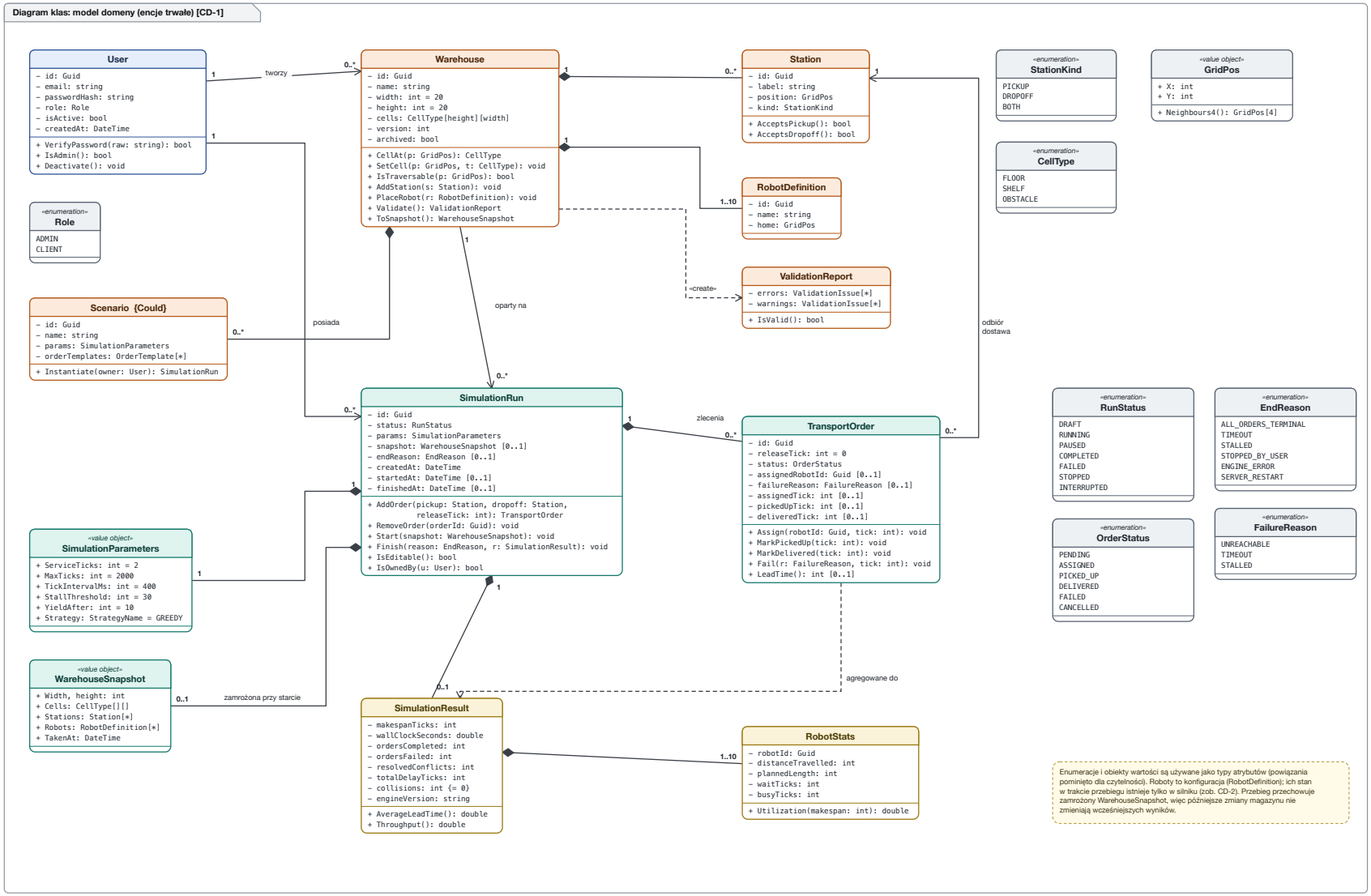
5.1 Serwisy między modułami

Moduły komunikują się wyłącznie przez poniższe serwisy. Tabela pokazuje, który moduł udostępnia serwis, kto z niego korzysta i jakie dane są przekazywane. Diagram klas CD-3 przedstawia te same serwisy z pełnymi sygnaturami.

Moduł dostawcy	Serwis / operacja	Używany przez	Przekazywane dane
Tożsamość i dostęp	`AuthService.Verify(token) → Principal`	kontrolery API, RunHub	JWT → id użytkownika + rola
Tożsamość i dostęp	`AccessPolicy.CanViewRun / CanControlRun`	RunService, ResultService, RunHub	Principal, SimulationRun → bool
Konfiguracja magazynu	`WarehouseService.Snapshot(id) → WarehouseSnapshot`	RunService (przy starcie)	zamrożona siatka, stacje, roboty
Konfiguracja magazynu	`WarehouseService.List / Get`	kontrolery API (przeglądanie)	WarehouseSummary, Warehouse
Przebiegi i zlecenia	`RunService.CreateRun / AddOrder / Start / Stop`	kontrolery API	polecenia klienta
Wykonanie symulacji	`RunManager.Launch(run) / Stop / Pause`	RunService	przebieg + migawka + zlecenia
Wykonanie symulacji	`RunManager.Snapshot(runId) → StateSnapshot`	RunService, RunHub	bieżący stan na żywo
Silnik symulacji	`SimulationEngine.Step() → TickUpdate, BuildResult()`	RunManager (przez ActiveRun)	stan ticku, metryki końcowe
Wykonanie symulacji	`LiveUpdatePublisher.Publish(runId, LiveMessage)`	RunManager → przeglądarki	snapshot / tick / run_status
Wyniki i analityka	`ResultService.Save(runId, result)`	RunManager (na końcu przebiegu)	SimulationResult
Wyniki i analityka	`ResultService.History / Get / Compare`	kontrolery API	RunSummary, SimulationResult
Trwałość danych	`IRepository<T>.Get / Save / List`	wszystkie serwisy	encje ↔ PostgreSQL

5.2 CD-1: model domeny

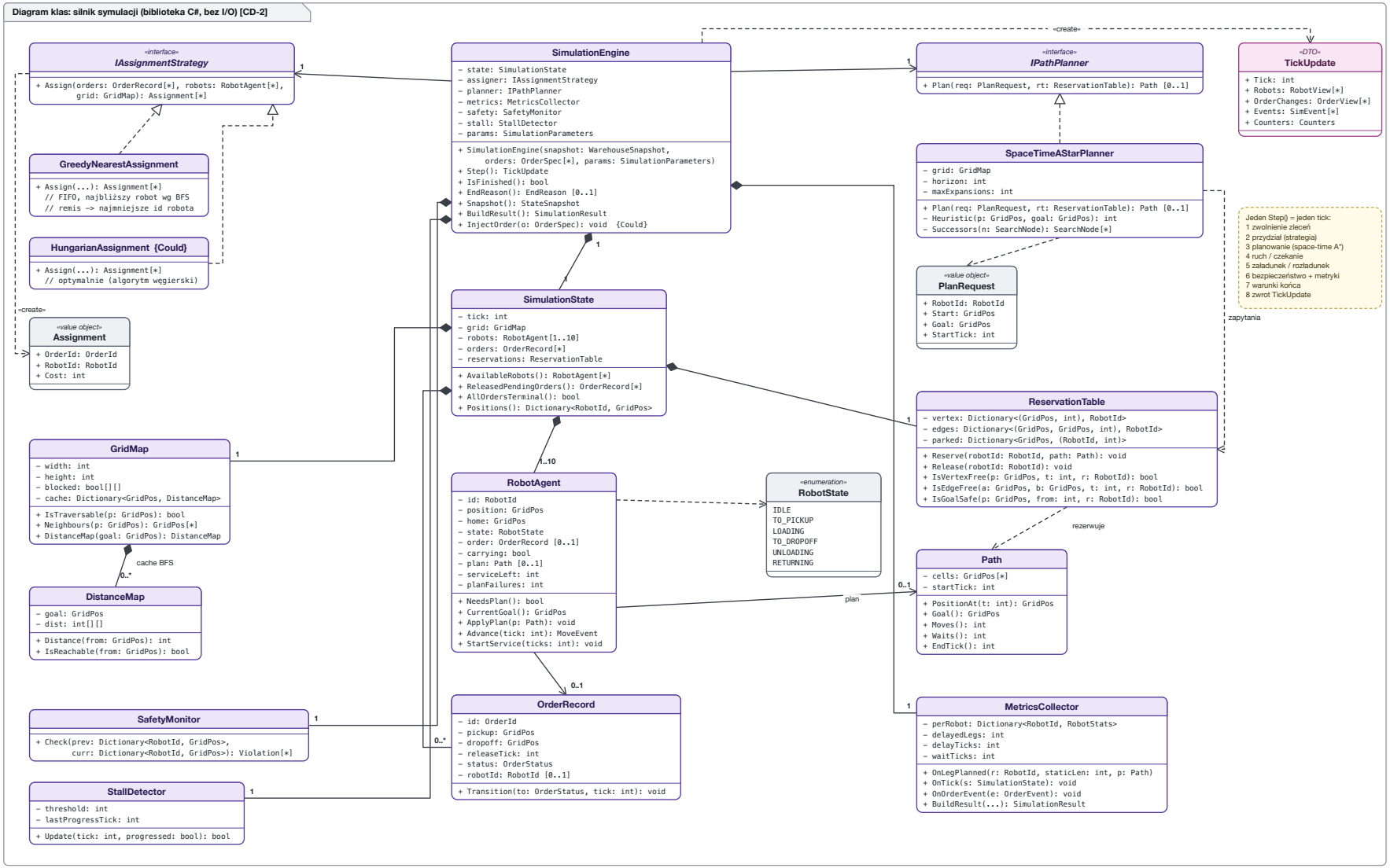
Model domeny zawiera encje trwałe. **Warehouse** składa się ze stacji i definicji robotów, sam się waliduje i tworzy **WarehouseSnapshot**. **SimulationRun** należy do jednego użytkownika i jednego magazynu oraz zawiera swoje **TransportOrder**, **SimulationParameters**, migawkę zamrożoną przy starcie, a na końcu **SimulationResult** ze statystykami robotów. Cykl życia przebiegów i zleceń opisują enumeracje RunStatus i OrderStatus.



Rysunek CD-1. Model domeny.

5.3 CD-2: silnik symulacji

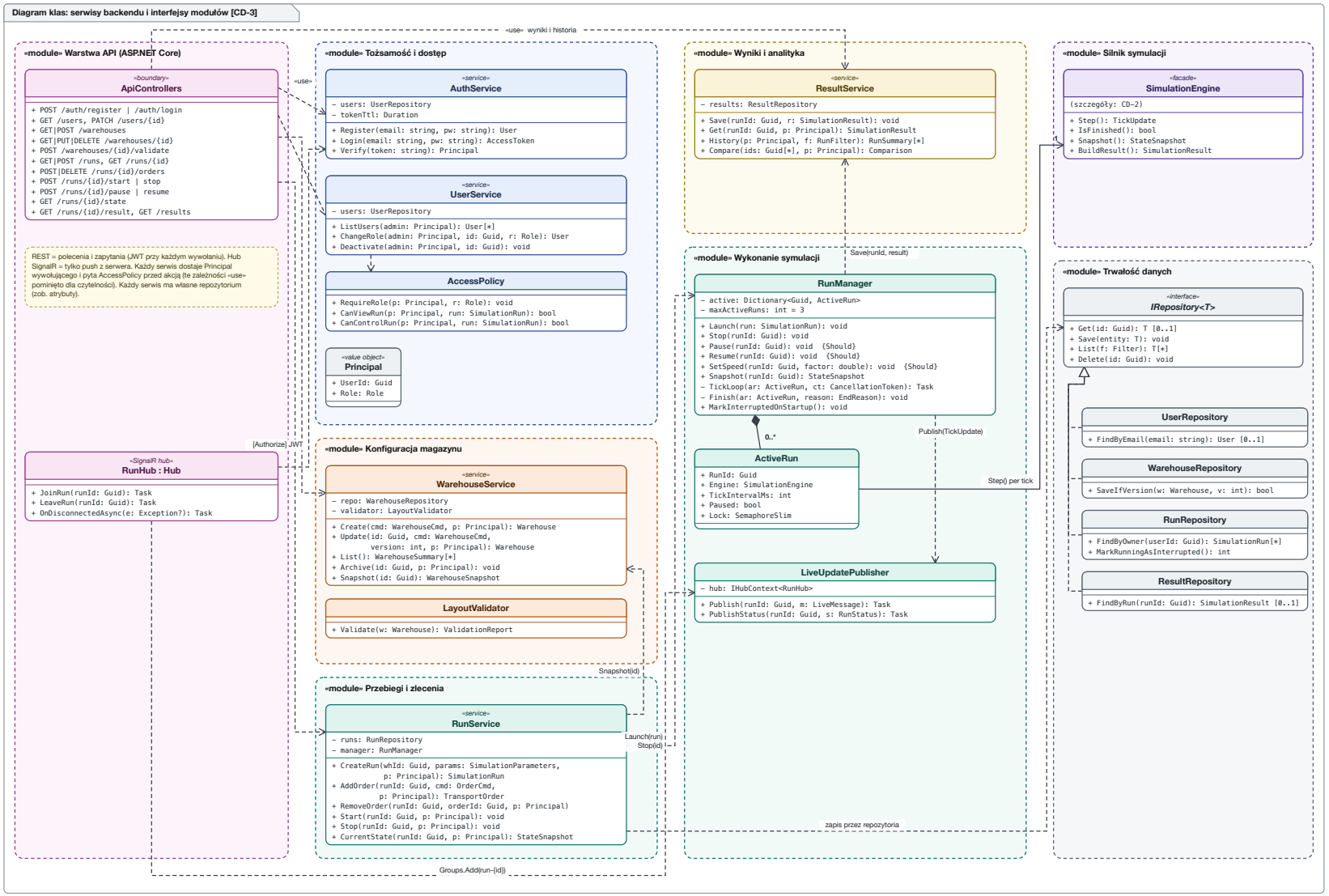
Silnik to czysty moduł (osobna biblioteka C#): otrzymuje migawkę, zlecenia i parametry oraz udostępnia metodę `Step()`. Przydział i planowanie tras są ukryte za interfejsami (`IAssignmentStrategy`, `IPathPlanner`), więc można dodawać i porównywać kolejne strategie. `SpaceTimeAStarPlanner` przeszukuje przestrzeń (x, y, t) i pyta `ReservationTable`, czy pole lub przejazd jest wolny w danym ticku; `SafetyMonitor` niezależnie sprawdza w każdym ticku, czy nie doszło do kolizji, a `StallDetector` kończy przebieg bez postępu.



Rysunek CD-2. Silnik symulacji.

5.4 CD-3: serwisy backendu i interfejsy modułów

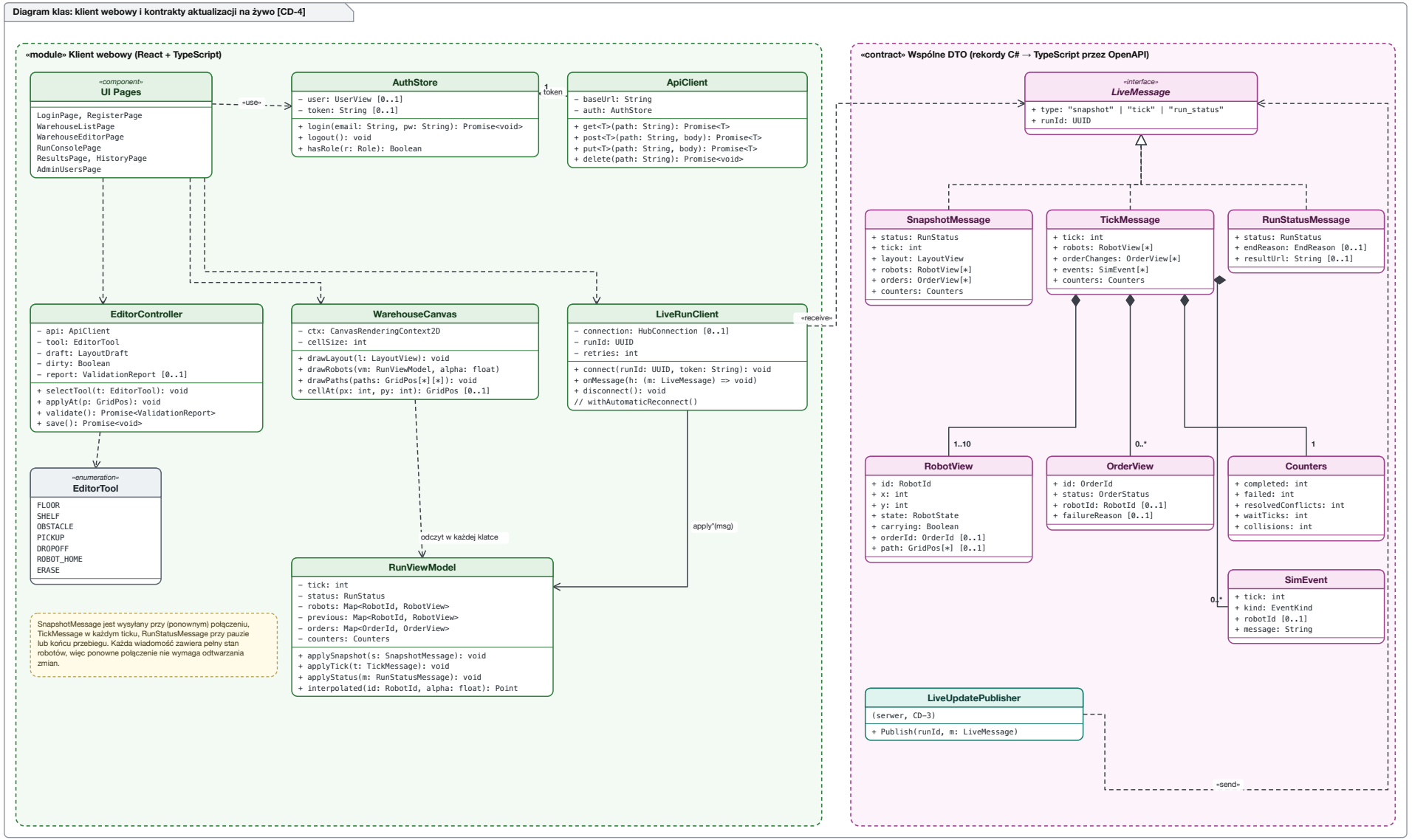
Każdy moduł udostępnia klasę «service»; warstwa API wywołuje wyłącznie serwisy, nigdy repozytoria. Każdy serwis otrzymuje `Principal` wywołującego i sprawdza uprawnienia przez `AccessPolicy`. `RunService` pobiera migawkę z `WarehouseService` i przekazuje przebieg do `RunManager`, który w każdym ticku wywołuje silnik, publikuje aktualizacje przez `LiveUpdatePublisher` (SignalR) i zapisuje wynik przez `ResultService`.



Rysunek CD-3. Serwisy backendu i interfejsy modułów.

5.5 CD-4: klient webowy i kontrakty aktualizacji na żywo

Klient webowy nie zawiera logiki symulacji: `LiveRunClient` odbiera wiadomości, `RunViewModel` przechowuje najnowszy stan, a `WarehouseCanvas` go rysuje, interpolując pozycje między tickami. Klasy wiadomości to wspólne kontrakty generowane ze schematu backendu, dzięki czemu obie strony używają tych samych typów.



Rysunek CD-4. Klient webowy i kontrakty aktualizacji na żywo.

6 Notacja i słownik pojęć

6.1 Notacja diagramów

- Tożsamość i dostęp
- Przebiegi, zlecenia i wykonanie symulacji
- Wyniki i analityka
- Klient webowy
- Konfiguracja magazynu
- Silnik symulacji
- Granica API / wspólne kontrakty
- Trwałość danych, enumeracje, obiekty wartości

Diagramy klas: wypełniony romb = kompozycja, pusty trójkąt (linia ciągła) = generalizacja, pusty trójkąt (linia przerywana) = realizacja interfejsu, przerywana strzałka otwarta = zależność («use», «create»), ciągła strzałka otwarta = asocjacja nawigowalna. `{Could}` / `{Should}` oznaczają elementy powiązane z historyjkami o niższym priorytecie. Nazwy klas, metod i atrybutów pozostają w języku angielskim, ponieważ są identyfikatorami w kodzie.

6.2 Słownik pojęć

Pojęcie	Znaczenie
Tick	Jeden dyskretny krok symulacji; każdy robot przesuwa się o jedno pole albo czeka.
Makespan	Liczba ticków do momentu, w którym ostatnie zlecenie osiągnęło stan końcowy (czas symulacji, nie czas rzeczywisty).
Konflikt wierzchołkowy	Dwa roboty zajęłyby to samo pole w tym samym ticku.
Konflikt krawędziowy	Dwa roboty zamieniłyby się polami w tym samym ticku.
Rozwiązany konflikt	Odcinek trasy, który musiał zostać opóźniony lub poprowadzony objazdem z powodu innych robotów (liczony w statystykach).
Kolizja	Faktyczny konflikt wierzchołkowy lub krawędziowy podczas wykonania; zawsze musi wynosić 0.
Tablica rezerwacji	Zapis tego, który robot zajmuje które pole / przejazd w którym ticku; używana przez planer.
Migawka	Zamrożona kopia konfiguracji magazynu i parametrów wykonana przy starcie przebiegu.
Zastój	Żaden robot się nie poruszył i żadne zlecenie nie zmieniło stanu przez K kolejnych ticków, mimo że praca pozostała.